SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	N. SRIKANTH	**Reg. No.:**	192411139
Section / Batch:	CSE	**Date:**	23/07/26

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:	**100 Marks**	

SECTION A – CONCEPT MAPPING

Q1. Explain how the concept of tight bound emerges when upper and lower bounds meet. Extend the map by adding links from Big-O and Big-Omega toward Big-Theta to show their relationship. Justify why Big-Theta is often preferred when the exact asymptotic growth is known. Interpret how the map helps distinguish between approximate and precise complexity descriptions.

Q2. Explain how dividing the original problem into smaller subproblems leads naturally to a recurrence relation. Extend the map by adding the parameters a and b as children of Number of

Subproblems and Problem Size Reduction respectively. Justify how these parameters influence the final complexity. Interpret how the concept map connects algorithm design structure with mathematical analysis.

Code of Conduct

I N.SRIKANTH(192411139) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Q1. ANSWER

Relationship Between Big-O, Big-Ω, and Big-Θ

The three asymptotic notations describe the growth rate of an algorithm from different perspectives:

- Big-O (O) gives an upper bound. It represents the maximum rate at which the running time or space requirement can grow.
- Big-Omega (Ω) gives a lower bound. It represents the minimum rate at which the running time or space requirement grows.
- Big-Theta (Θ) gives a tight bound. It specifies that the running time grows at exactly the same asymptotic rate from both above and below.

How the Tight Bound Emerges

A tight bound is obtained when the upper and lower bounds coincide.

If

$$T(n) = O(f(n))$$

and

$$T(n) = \Omega(f(n)),$$

then both bounds describe the same growth function. Therefore,

$$T(n) = \Theta(f(n)).$$

This means there exist positive constants c_1, c_2, and n_0 such that

$$c_1 f(n) \leq T(n) \leq c_2 f(n), n \geq n_0.$$

Hence, Big-Theta combines the information provided by both Big-O and Big-Omega into a single, exact asymptotic description.

Why Big-Theta Is Preferred

Big-Theta is often preferred because:

- It gives the exact asymptotic growth rate, rather than only a maximum or minimum.
- It removes ambiguity about how efficiently an algorithm scales.
- It allows meaningful comparison between algorithms that have the same true growth rate.
- It provides a complete characterization of performance for large input sizes.

For example,

- Saying an algorithm is $O(n^2)$only tells us it does not grow faster than quadratic time.
- Saying it is $\Theta(n^2)$tells us it grows exactly at a quadratic rate asymptotically.

Thus, Big-Theta conveys more precise information than Big-O or Big-Omega alone.

Approximate vs. Precise Complexity Descriptions

The relationship map helps distinguish between different levels of precision:

Notation	Description	Type of Information
Big-O (O)	Upper bound	Approximate (maximum growth)
Big-Omega (Ω)	Lower bound	Approximate (minimum growth)
Big-Theta (Θ)	Tight bound	Precise (exact asymptotic growth)

- Big-O is useful when only the worst-case limit is known.
- Big-Omega is useful when only the minimum guaranteed growth is known.
- Big-Theta is used when both bounds match, providing the most accurate asymptotic description.

Conclusion

A tight bound arises when an algorithm has both the same upper bound and lower bound, resulting in Big-Theta notation:

$$\boxed{O(f(n)) \cap \Omega(f(n)) = \Theta(f(n))}$$

Therefore, Big-Theta is the preferred notation whenever the exact asymptotic growth rate is known, while Big-O and Big-Omega provide only approximate upper and lower bounds, respectively.

Q2. ANSWER

Divide and Conquer and the Formation of a Recurrence Relation

In the Divide and Conquer strategy, a large problem is repeatedly divided into smaller subproblems of the same type. Each subproblem is solved recursively, and the individual solutions are then combined to obtain the final result. Since the same computation is performed on progressively smaller inputs, the running time naturally depends on the running time of the smaller instances. This dependency is expressed using a recurrence relation.

The general form of the recurrence is:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where:

- a= Number of subproblems generated at each recursive step.
- b= Factor by which the problem size is reduced.
- $f(n)$= Time required to divide the problem and combine the solutions.

Influence of Parameters a and b on Complexity

1. Parameter a (Number of Subproblems)

The parameter $a$determines how many recursive calls are made at each level of recursion.

- A larger value of $a$increases the total amount of recursive work.
- More recursive calls generally result in a larger recursion tree and higher time complexity.

Example:

- $a = 2$: Two recursive calls (Merge Sort)
- $a = 3$: Three recursive calls (Karatsuba-like variations)

Thus, increasing $a$tends to increase the algorithm's running time.

2. Parameter b (Problem Size Reduction)

The parameter $b$determines how much the input size is reduced in each recursive call.

- A larger value of $b$reduces the input size more rapidly.
- Faster reduction decreases the recursion depth, leading to fewer recursive levels.

The recursion depth is approximately

$$\log_b n.$$

For example:

- $b = 2$: Depth is $\log_2 n$
- $b = 4$: Depth is $\log_4 n$, which is smaller

Hence, a larger $b$generally reduces the overall running time.

Connection Between Algorithm Design and Mathematical Analysis

The concept map illustrates how the structure of a Divide and Conquer algorithm is directly translated into a mathematical recurrence relation.

- The Divide Problem step determines the problem size reduction (b).
- The number of recursive calls determines the parameter (a).
- The Combine Results step contributes the non-recursive work $f(n)$.

Together, these components form the recurrence

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

which can then be solved using methods such as the Substitution Method, Recursion Tree Method, or Master Theorem to determine the algorithm's asymptotic complexity.

Interpretation

The concept map establishes a clear relationship between algorithm design and complexity analysis. It shows that every Divide and Conquer algorithm can be modeled by a recurrence relation, where the parameters $a$and $b$describe the recursive structure of the algorithm. By analyzing these parameters along with the non-recursive work $f(n)$, the overall time complexity can be derived. Thus, the map serves as a bridge between the practical design of recursive algorithms and their theoretical performance analysis.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Marks Evaluation:

Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____